VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI - 590018, KARNATAKA

BAI702: Machine Learning II Mini Project Report On

"DIABETES RISK PREDICTOR USING MACHINE LEARNING"

Submitted in partial fulfillment of the requirements for the Seventh Semester of

Bachelor of Engineering in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Submitted by

Soumya Math (1AM23CI190)

Tejaswini N H (1AM23CI169)

Sushmita G V (1AM23CI159)

Teppala Shruti (1AM23CI170)

Under the Guidance of

Prof. Bhavya K Bharathan

Assistant Professor, Dept. of CSE(AIML)

Department of

Computer Science & Engineering

(Artificial Intelligence and Machine Learning)

AMC ENGINEERING COLLEGE

Bannerghatta Road, Bangalore-560083

2026-27

AMC ENGINEERING COLLEGE

Bannerghatta Road, Bangalore-560083

Department of

Computer Science & Engineering

(Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that the Mini Project Report entitled "**Diabetes Risk Predictor using Machine Learning**" is a bonafide work carried out by **Soumya Math (1AM23CI190), Tejaswini N H (1AM23CI169), Sushmita G V (1AM23CI159), Teppala Shruti (1AM23CI170)**, of the VII Semester, **Bachelor of Engineering** in **Computer Science and Engineering (Artificial Intelligence & Machine Learning)**, AMC Engineering College, Bengaluru, during the academic year 2026-27 as part of the course BAI702 – Machine Learning 2 under the guidance of Prof. Bhavya K Bharathan, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning). The work has been completed as a part of the course requirements and has been evaluated as per the prescribed academic guidelines.

Prof. Bhavya K Bharathan
Assistant Professor & Project Guide
Department of CSE (AIML), AMCEC

DECLARATION

We, the undersigned students of the seventh semester, Bachelor of Engineering, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), AMC Engineering College, Bengaluru, declare that our mini project work entitled “Diabetes Risk Predictor using Machine Learning” is a bonafide work of ours. This project is neither a copy nor a modification of any other engineering project. We also declare that this project has not been submitted to any other university in the past and will not be submitted to any other university in the future.

Name	USN	Signature
Soumya Math	1AM23CI190	
Tejaswini N H	1AM23CI169	
Sushmita G V	1AM23CI159	
Teppala Shruti	1AM23CI170	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to AMC Engineering College, Bengaluru, for providing the opportunity and the facilities to carry out this mini project as part of the course BAI702 – Machine Learning 2.

We express our sincere thanks to Dr. Nandeeswar S B, Professor and HOD, Department of CSE(AIML), for his support and encouragement.

We are grateful to our guide Prof. Bhavya K Bharathan, Assistant Professor, Department of CSE(AIML), for the valuable guidance, suggestions and continuous support throughout this mini project.

We also thank our faculty members, classmates and everyone who directly or indirectly supported us in completing this work successfully.

Finally, we thank our parents and family members for their constant encouragement and support.

Soumya Math	1AM23CI190
Tejaswini N H	1AM23CI169
Sushmita G V	1AM23CI159
Teppala Shruti	1AM23CI170

ABSTRACT

Diabetes is a chronic disease whose complications can be reduced substantially when it is detected early. This project builds and compares five supervised machine learning classifiers (Logistic Regression, K-Nearest Neighbours, Decision Tree, Random Forest and Gradient Boosting) to predict diabetes from eight routine diagnostic measurements in the Pima Indians Diabetes dataset from Kaggle (768 records, 34.9% positive class). Physiologically impossible zero values hidden in five columns were treated as missing and imputed, outliers were capped with the IQR rule, and features were standardised, all fitted on training data only. Models were trained on a stratified 80/20 split with explicitly stated hyperparameters and evaluated using Accuracy, Precision, Recall, F1-Score, ROC-AUC and confusion matrices, and additionally with 5-fold stratified cross-validation. The Decision Tree scored highest on the single 154-sample test set (79.87% accuracy, 0.825 ROC-AUC), but cross-validation showed this to be unstable (CV ROC-AUC 0.785 ± 0.038); Logistic Regression (0.839 ± 0.017) and Random Forest (0.836 ± 0.020) were statistically tied for best. Random Forest was selected for its stable discrimination, lowest false-positive count (13) and interpretable feature importances, with Glucose, BMI and Age as the leading predictors. The selected model was deployed as a browser-based Diabetes Risk Predictor whose predictions match scikit-learn exactly on all 154 test patients. In conclusion, ensemble and regularised linear models separate diabetic from non-diabetic patients well (ROC-AUC of about 0.84), but recall (57–65%) is too low for standalone screening, so threshold tuning is the key next step.

CONTENTS

CHAPTER	TITLE	PAGE
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Scope	1
CHAPTER 2	**DATASET & PREPROCESSING**	**2**
2.1	Dataset Source	2
2.2	Feature Description (Table 2.1)	2
2.3	Data Inspection	2
2.4	Handling Missing/Invalid Values, Duplicates, and Outliers	3
2.5	Exploratory Data Analysis (EDA)	3
2.6	Train-Test Split	5
2.7	Feature Scaling	5
CHAPTER 3	**MODEL DEVELOPMENT**	**6**
3.1	Logistic Regression	6
3.2	K-Nearest Neighbours (KNN)	6
3.3	Decision Tree	6
3.4	Random Forest	7
3.5	Gradient Boosting	7
3.6	Training Process Summary	7
CHAPTER 4	**EVALUATION METRICS**	**9**
4.1	Accuracy	9
4.2	Precision	9
4.3	Recall (Sensitivity)	9
4.4	F1-Score	9
4.5	ROC-AUC	9
4.6	Confusion Matrix (Table 4.1)	9
CHAPTER 5	**RESULTS & COMPARISON**	**10**
5.1	Comparison Table (All Models × All Metrics) (Table 5.1)	10
5.2	Bar Chart Comparisons	10
5.3	Confusion Matrices — All Models (Table 5.2)	10
5.4	ROC Curves (Overlaid)	11
5.5	Feature Importance (Random Forest)	11
5.6	Cross-Validated Performance (Table 5.3)	12
CHAPTER 6	**ANALYSIS & INTERPRETATION**	**13**
6.1	Best and Worst Model per Metric	13
6.2	Why the Rankings Differ between the Test Set and Cross-Validation	13
6.3	Why Ensemble Models Tend to Perform Better	13
6.4	Interpretation for Screening	13
CHAPTER 7	**BEST MODEL SELECTION**	**14**
7.1	Justification of the Selected Model	14
7.2	Deployment: Diabetes Risk Predictor	14
7.3	Technology Stack	15
7.4	Flowchart and Architecture	15
7.5	Core Module: Running the Random Forest in the Browser	16

7.6	User Interface	17
7.7	Testing and Verification	18
CHAPTER 8	**ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE**	**20**
8.1	Advantages	20
8.2	Limitations	20
8.3	Future Scope	20
CHAPTER 9	**CONCLUSION**	**21**
	REFERENCES	**22**

LIST OF FIGURES

FIGURE	TITLE	PAGE
2.1	Histograms of All Features	4
2.2	Class Distribution of the Target Variable	4
2.3	Correlation Heatmap of Features and Target	5
2.4	Boxplots Used for Outlier Inspection	5
3.1	Trained Decision Tree (Top Three Levels)	7
5.1	Bar Chart Comparison of All Models across All Metrics	10
5.2	Confusion Matrices of All Five Models	11
5.3	ROC Curves of All Models (Overlaid)	11
5.4	Feature Importance of the Selected Random Forest	12
5.5	Cross-Validated ROC-AUC per Model	12
7.1	Flowchart of the Diabetes Risk Predictor	16
7.2	Architecture of the Diabetes Risk Predictor	16
7.3	Diabetes Risk Predictor: High-Risk and Low-Risk Examples	18
7.4	Browser Model vs scikit-learn Random Forest	19

LIST OF TABLES

TABLE	TITLE	PAGE
2.1	Feature Description of the Pima Indians Diabetes Dataset	2
2.2	Summary Statistics of the Raw Data	3
2.3	Invalid Zero Values Detected	3
3.1	Hyperparameters Used for Every Model	8
4.1	Confusion Matrix Structure (2×2)	9
5.1	Comparison Table (All Models × All Metrics)	10
5.2	Confusion Matrix Values for All Models	10
5.3	5-Fold Stratified Cross-Validation Results	12
6.1	Best and Worst Model for Each Metric	13
7.1	Technology Stack	15
7.2	Test Cases for the Diabetes Risk Predictor	18

CHAPTER 1
INTRODUCTION

1.1 Background

Diabetes mellitus is a chronic condition in which the body cannot regulate blood glucose properly, either because of insufficient insulin production or because of insulin resistance. Left undiagnosed it leads to serious complications such as cardiovascular disease, kidney failure, nerve damage and blindness. Because the early stages are often symptom-free, screening based on simple clinical measurements such as plasma glucose, body-mass index, blood pressure and age is valuable.

Machine learning can learn patterns from historical patient records and turn these routine measurements into a risk prediction. Comparing different families of algorithms (linear, distance-based, tree-based and ensemble) on the same data shows which assumptions suit medical tabular data best and how much extra benefit ensembles provide.

1.2 Problem Statement

Given eight diagnostic measurements for a female patient (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age), predict whether she has diabetes (Outcome = 1) or not (Outcome = 0). This is a supervised binary classification problem on a small, mildly imbalanced and imperfect (hidden missing values) dataset.

1.3 Objectives

- To inspect, clean and explore the Pima Indians Diabetes dataset, including detecting hidden missing values.
- To build a leak-free preprocessing pipeline (imputation, outlier capping, scaling) and a stratified train–test split.
- To train five classifiers (Logistic Regression, KNN, Decision Tree, Random Forest, Gradient Boosting) with explicitly stated hyperparameters.
- To evaluate all models using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices, supported by 5-fold cross-validation.
- To select the best model with a justification that goes beyond accuracy, and to identify the most influential features.
- To deploy the selected model as a browser-based Diabetes Risk Predictor and to verify that it reproduces the Python model exactly.

1.4 Scope

The study is limited to one publicly available tabular dataset and to five scikit-learn classifiers. Hyperparameters are set manually with regularising values (no exhaustive grid search). A small browser application, the Diabetes Risk Predictor, is built to demonstrate the selected model. The project is an academic comparison of models and is not a clinically validated diagnostic tool.

CHAPTER 2
DATASET & PREPROCESSING

2.1 Dataset Source

The dataset is the **Pima Indians Diabetes Database**, originally collected by the US National Institute of Diabetes and Digestive and Kidney Diseases and published on Kaggle (link below). It contains diagnostic records of 768 women, all at least 21 years old and of Pima Indian heritage.

Dataset link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

Note: the identical 768-row CSV (without a header row) was also retrieved from https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv and the standard column names were added; results in this report were computed on that file.

2.2 Feature Description (Table 2.1)

Table 2.1: Feature description

Feature	Description	Unit
Pregnancies	Number of times pregnant	count
Glucose	Plasma glucose concentration after a 2-hour oral glucose tolerance test	mg/dL
BloodPressure	Diastolic blood pressure	mm Hg
SkinThickness	Triceps skin-fold thickness	mm
Insulin	2-hour serum insulin	µU/mL
BMI	Body-mass index (weight / height²)	kg/m²
DiabetesPedigreeFunction	Score estimating diabetes likelihood from family history	score
Age	Age of the patient	years
Outcome (target)	1 = diabetic, 0 = non-diabetic	binary

2.3 Data Inspection

The dataset has **768 rows and 9 columns** (8 predictors and 1 target). *df.info()* shows that all columns are numeric (7 integer, 2 float) and *df.isnull().sum()* returns zero for every column; *df.duplicated().sum()* also returns zero. The class distribution is 500 non-diabetic (65.1%) and 268 diabetic (34.9%) records.

Table 2.2: Summary statistics of the raw data (df.describe())

Feature	Mean	Std	Min	Median	Max
Pregnancies	3.85	3.37	0	3.00	17
Glucose	120.89	31.97	0	117.00	199
BloodPressure	69.11	19.36	0	72.00	122
SkinThickness	20.54	15.95	0	23.00	99
Insulin	79.80	115.24	0	30.50	846
BMI	31.99	7.88	0	32.00	67.10
DiabetesPedigreeFunction	0.47	0.33	0.08	0.37	2.42
Age	33.24	11.76	21	29.00	81
Outcome	0.35	0.48	0	0	1

Although there are no null values, the *Min* column reveals a hidden problem: Glucose, BloodPressure, SkinThickness, Insulin and BMI contain zeros, which are physiologically impossible (a living patient cannot have zero blood pressure or BMI). These zeros are placeholders for missing measurements (Table 2.3). Zero is a valid value for Pregnancies, so that column was left unchanged.

Table 2.3: Invalid zero values detected

Feature	Zero count	% of 768 records
Glucose	5	0.7%
BloodPressure	35	4.6%
SkinThickness	227	29.6%
Insulin	374	48.7%
BMI	11	1.4%

2.4 Handling Missing/Invalid Values, Duplicates, and Outliers

- **Missing / invalid values:** zeros in the five columns above were replaced by NaN and then imputed with the **median** of the training set. The median is preferred over the mean because Insulin and SkinThickness are right-skewed and contain extreme values.
- **Duplicates:** none were found, so no rows were removed.
- **Outliers:** the boxplots (Figure 2.4) show extreme values, notably Insulin (up to 846), SkinThickness (99), BloodPressure and DiabetesPedigreeFunction. Because the dataset is small, rows were not deleted; instead, values were **capped** to the interval below, with Q1, Q3 computed on the training set only:

$$[\,Q_1 - 1.5 \times IQR,\;\; Q_3 + 1.5 \times IQR\,], \quad IQR = Q_3 - Q_1$$

All imputation and capping statistics were computed on the training data and then applied to the test data, which prevents **data leakage**.

2.5 Exploratory Data Analysis (EDA)

Figure 2.1 shows the distribution of every feature. Glucose, BloodPressure and BMI are roughly bell-shaped; Pregnancies, Insulin, DiabetesPedigreeFunction and Age are strongly right-skewed. Because the invalid zeros were converted to NaN before plotting, the histograms

show only valid measurements, and the long right tails of Insulin and SkinThickness are clearly visible.

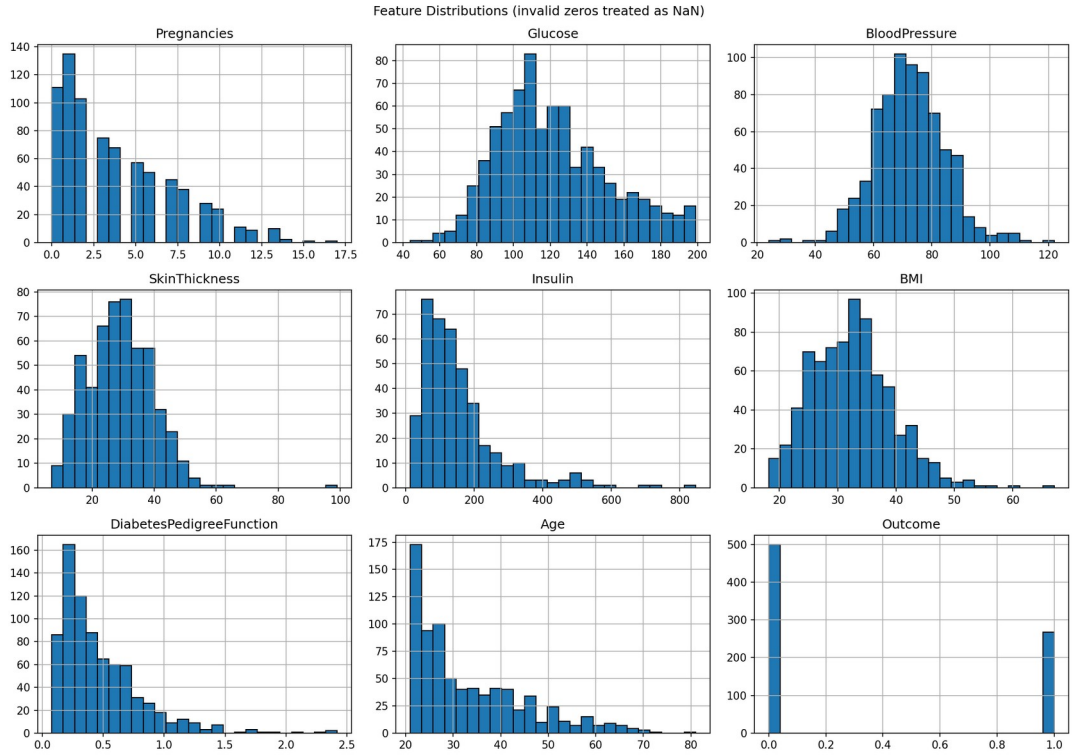


Figure 2.1: Histograms of all features

Figure 2.2 shows that the classes are mildly imbalanced (about 2 : 1). A model that always predicts “non-diabetic” would already reach 65.1% accuracy, so accuracy alone is a weak measure and precision, recall, F1 and ROC-AUC are reported as well. Stratified splitting is used to keep the ratio identical in train and test sets.

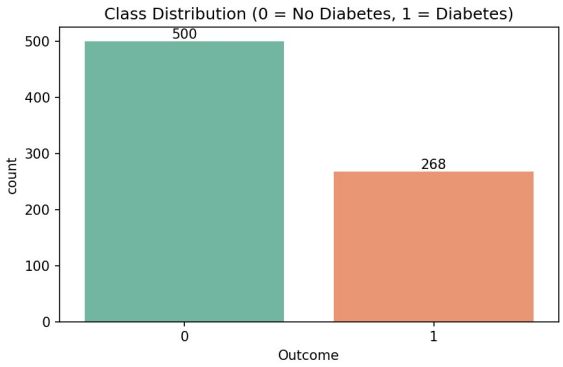


Figure 2.2: Class distribution of the target variable

The correlation heatmap (Figure 2.3) shows that Glucose has the strongest correlation with Outcome (0.49), followed by BMI (0.31), Insulin (0.30), SkinThickness (0.26) and Age (0.24). Among predictors, SkinThickness–BMI (0.65), Insulin–Glucose (0.58) and Age–Pregnancies (0.54) are moderately correlated; this is not severe enough to drop features, and tree-based models are insensitive to it.

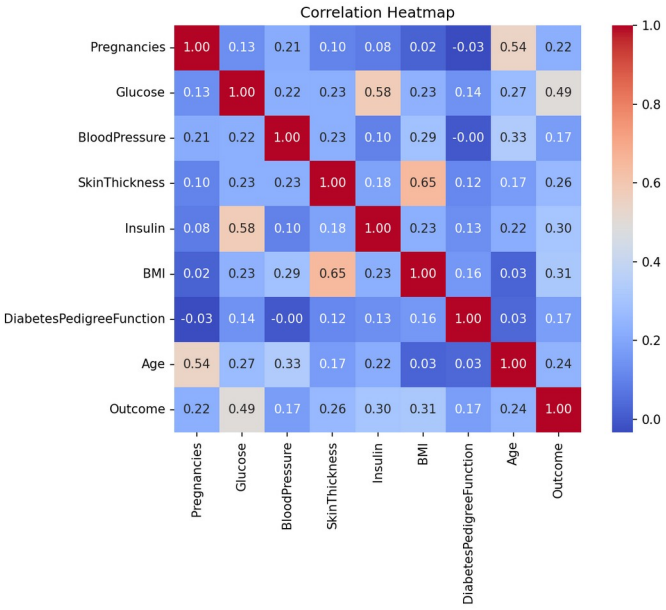


Figure 2.3: Correlation heatmap of features and target

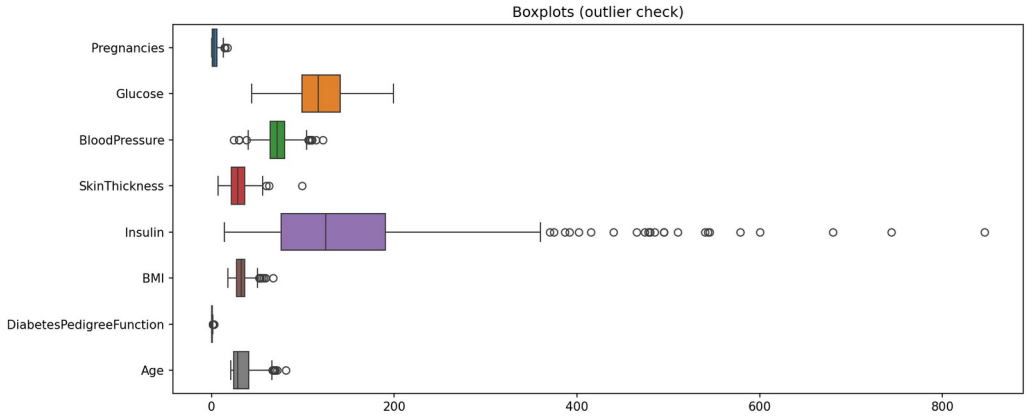


Figure 2.4: Boxplots used for outlier inspection

2.6 Train-Test Split

The data was divided with *train_test_split(test_size = 0.2, random_state = 42, stratify = y)*. This gives **614 training** and **154 test** samples; the test set contains 100 non-diabetic and 54 diabetic patients, and both sets keep the original 65.1 : 34.9 class ratio. A fixed random_state makes every number in this report reproducible.

2.7 Feature Scaling

Features were standardised with **StandardScaler** (zero mean, unit variance), fitted on the training set only:

$$z = \frac{x - \mu}{\sigma}$$

Scaling is essential for Logistic Regression (regularisation penalises coefficients of large-valued features unfairly and gradient-based solvers converge faster) and for KNN (distance calculations would otherwise be dominated by Insulin and Glucose, which have much larger ranges than DiabetesPedigreeFunction). Tree-based models are invariant to monotonic scaling, but the same scaled matrix was used for all models for consistency.

CHAPTER 3
MODEL DEVELOPMENT

Five classifiers representing different learning principles were selected. Each is described below with how it works, its strengths and weaknesses, and the hyperparameters used. No hyperparameter was left silently at its default; defaults that were kept deliberately are marked as such.

3.1 Logistic Regression

Logistic Regression models the probability of the positive class by passing a weighted sum of the features through the sigmoid function and learns the weights by minimising the (L2-regularised) log-loss:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x} + b)}}$$

Pros: fast, simple, well-calibrated probabilities, coefficients are interpretable, little overfitting on small data. **Cons:** assumes a linear decision boundary and cannot capture feature interactions unless engineered.

Hyperparameters: C = 1.0 (inverse regularisation strength), L2 penalty (default, kept deliberately), solver = lbfgs, max_iter = 1000 (raised from the default 100 to guarantee convergence), random_state = 42.

3.2 K-Nearest Neighbours (KNN)

KNN stores the training set and classifies a new patient by a (distance-weighted) vote among the k most similar training patients, using Euclidean distance:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2}$$

Pros: no training phase, captures non-linear boundaries. **Cons:** sensitive to scaling and to irrelevant features, slow at prediction time on big data, and suffers when many features are noisy or imputed.

Hyperparameters: n_neighbors = 7 (odd value to avoid ties, large enough to smooth noise), weights = distance (closer neighbours count more), metric = minkowski with p = 2 (Euclidean).

3.3 Decision Tree

A decision tree recursively splits the data on the feature and threshold that most reduces node impurity (here the Gini index) until a stopping rule is reached; each leaf predicts the majority class:

$$Gini = 1 - \sum_{k=1}^{K} p_k^2$$

Pros: easy to visualise and explain (Figure 3.1), no scaling needed, captures non-linear interactions. **Cons:** high variance — small changes in data can produce a very different tree — and prone to overfitting unless depth is restricted.

Hyperparameters: criterion = gini, max_depth = 4, min_samples_split = 10, min_samples_leaf = 5 (these three limit tree size to reduce overfitting), random_state = 42.

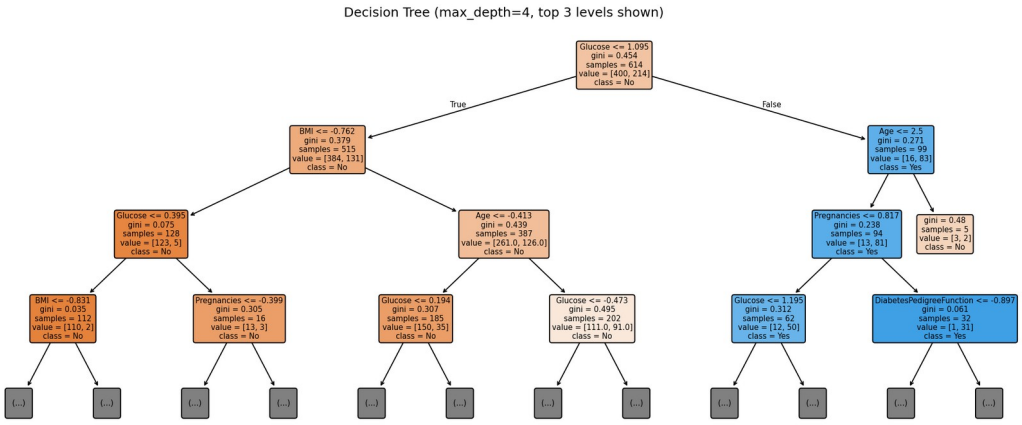


Figure 3.1: Trained Decision Tree (top three levels shown)

3.4 Random Forest

Random Forest is a bagging ensemble. It trains many decorrelated decision trees, each on a bootstrap sample of the data and using only a random subset of features at every split, and combines them by majority vote (or averaged probabilities):

$$\hat{y} = \text{majority vote}\{h_1(\mathbf{x}),\ h_2(\mathbf{x}),\ \ldots,\ h_B(\mathbf{x})\}$$

Pros: much lower variance than a single tree, robust to outliers and noise, handles non-linearity, provides feature importances. **Cons:** less interpretable than one tree, larger and slower, and probabilities are not always well calibrated.

Hyperparameters: n_estimators = 300 trees, max_depth = 6, min_samples_split = 5, min_samples_leaf = 2, max_features = sqrt ($\sqrt{8} \approx 3$ features tried per split), bootstrap = True (default, kept deliberately), random_state = 42.

3.5 Gradient Boosting

Gradient Boosting builds shallow trees sequentially; each new tree fits the errors (gradients of the log-loss) of the current ensemble, and its contribution is shrunk by the learning rate η:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x})$$

Pros: often the highest accuracy on tabular data, models complex interactions, built-in feature importance. **Cons:** more hyperparameters to tune, sensitive to noise and overfitting if over-trained, and sequential training cannot be parallelised.

Hyperparameters: n_estimators = 200, learning_rate = 0.05 (small steps, compensated by more trees), max_depth = 3 (weak learners), subsample = 0.8 (stochastic boosting for regularisation), loss = log_loss (default), random_state = 42.

3.6 Training Process Summary

Table 3.1 lists the hyperparameters used for every model. The steps of the training process follow the table.

Table 3.1: Hyperparameters used for every model

Model	Hyperparameters
Logistic Regression	C = 1.0, penalty = L2 (default), solver = lbfgs, max_iter = 1000, random_state = 42
K-Nearest Neighbours	n_neighbors = 7, weights = distance, metric = minkowski, p = 2 (Euclidean)
Decision Tree	criterion = gini, max_depth = 4, min_samples_split = 10, min_samples_leaf = 5, random_state = 42
Random Forest	n_estimators = 300, max_depth = 6, min_samples_split = 5, min_samples_leaf = 2, max_features = sqrt, bootstrap = True, random_state = 42
Gradient Boosting	n_estimators = 200, learning_rate = 0.05, max_depth = 3, subsample = 0.8, loss = log_loss, random_state = 42

1. Load the data and convert invalid zeros to NaN.
2. Split into 80 % training and 20 % test data (stratified, random_state = 42).
3. Compute medians, IQR bounds and scaling statistics on the training set only and apply them to both sets.
4. Fit each of the five models on the scaled training data with the hyperparameters in Table 3.1.
5. Predict class labels and probabilities on the test set and compute all metrics.
6. Repeat the evaluation with 5-fold stratified cross-validation (shuffle = True, random_state = 42), where imputation, capping and scaling are placed in a scikit-learn Pipeline and re-fitted inside each training fold, so no information leaks from the validation fold.

Hyperparameters were chosen manually using regularising values suited to a small dataset (shallow trees, small learning rate); an automated grid search was not performed and is listed as future scope.

CHAPTER 4
EVALUATION METRICS

All metrics are computed from the confusion matrix. In this project the positive class (1) is "diabetic". Let TP = true positives, TN = true negatives, FP = false positives and FN = false negatives.

4.1 Accuracy

The fraction of all predictions that are correct. It can be misleading when classes are imbalanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

4.2 Precision

Of all patients predicted diabetic, the fraction that truly are. High precision means few false alarms.

$$\text{Precision} = \frac{TP}{TP + FP}$$

4.3 Recall (Sensitivity)

Of all truly diabetic patients, the fraction that the model finds. High recall means few missed cases.

$$\text{Recall } (TPR) = \frac{TP}{TP + FN}$$

4.4 F1-Score

The harmonic mean of precision and recall; it is high only when both are high.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.5 ROC-AUC

The ROC curve plots the true-positive rate (recall) against the false-positive rate (FPR) for every possible threshold. The area under it (AUC) measures how well the model ranks a diabetic patient above a non-diabetic one: 0.5 is random guessing and 1.0 is perfect separation. Unlike accuracy, it does not depend on the 0.5 threshold or on class balance.

$$FPR = \frac{FP}{FP + TN} \qquad AUC = \int_0^1 TPR \; d(FPR)$$

4.6 Confusion Matrix (Table 4.1)

A confusion matrix cross-tabulates the actual class against the predicted class (Table 4.1). A false positive is a healthy patient flagged as diabetic (an unnecessary follow-up test); a false negative is a diabetic patient who is missed, the more serious error in screening.

Table 4.1: Structure of the confusion matrix

	Predicted: No (0)	**Predicted: Yes (1)**
Actual: No (0)	True Negative (TN)	False Positive (FP)
Actual: Yes (1)	False Negative (FN)	True Positive (TP)

CHAPTER 5
RESULTS & COMPARISON

5.1 Comparison Table (All Models × All Metrics) (Table 5.1)

Table 5.1 compares all five models on the 154-sample test set (best value in each column in bold).

Table 5.1: Test-set metrics for all models

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7143	0.6087	0.5185	0.5600	0.8085
KNN	0.7532	0.6667	0.5926	0.6275	0.7904
Decision Tree	**0.7987**	**0.7091**	**0.7222**	**0.7156**	**0.8248**
Random Forest	0.7468	0.6829	0.5185	0.5895	0.8150
Gradient Boosting	0.7532	0.6600	0.6111	0.6346	0.8120

5.2 Bar Chart Comparisons

Figure 5.1 shows all five metrics for every model side by side.

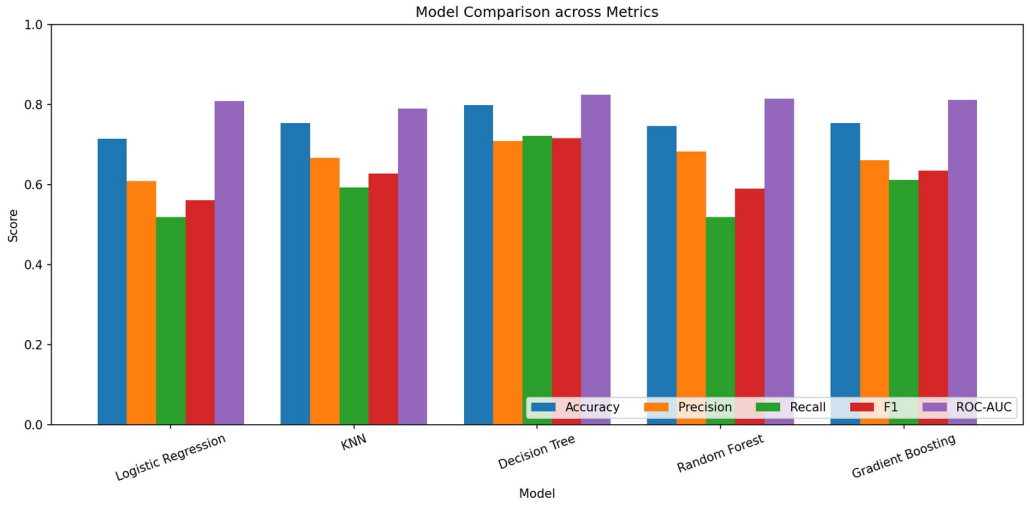


Figure 5.1: Bar chart comparison of all models across all metrics (test set)

5.3 Confusion Matrices — All Models (Table 5.2)

Table 5.2: Confusion matrix counts on the test set (100 non-diabetic, 54 diabetic)

Model	TN	FP	FN	TP
Logistic Regression	82	18	26	28
KNN	84	16	22	32
Decision Tree	84	16	15	39
Random Forest	87	13	26	28
Gradient Boosting	83	17	21	33

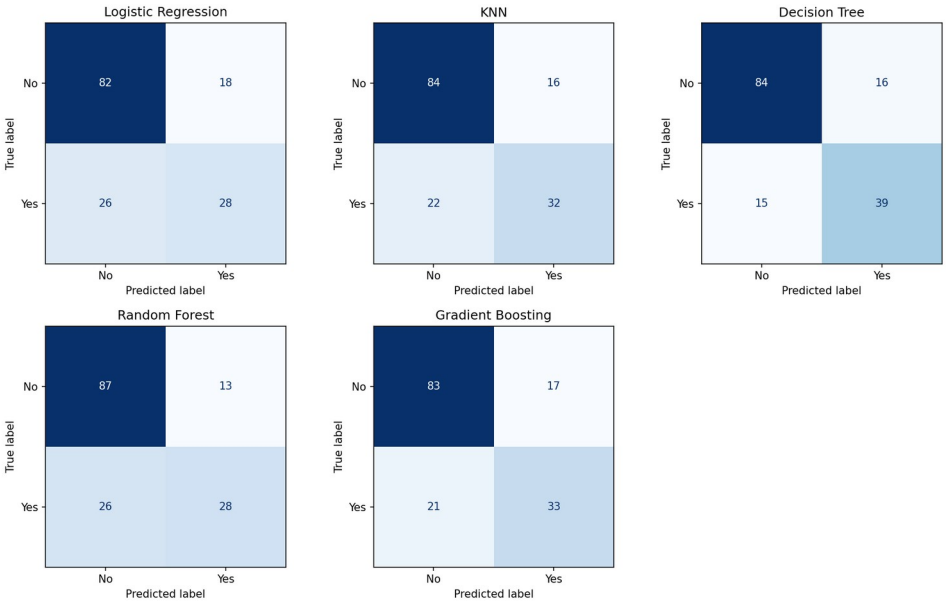


Figure 5.2: Confusion matrices of all five models (test set)

5.4 ROC Curves (Overlaid)

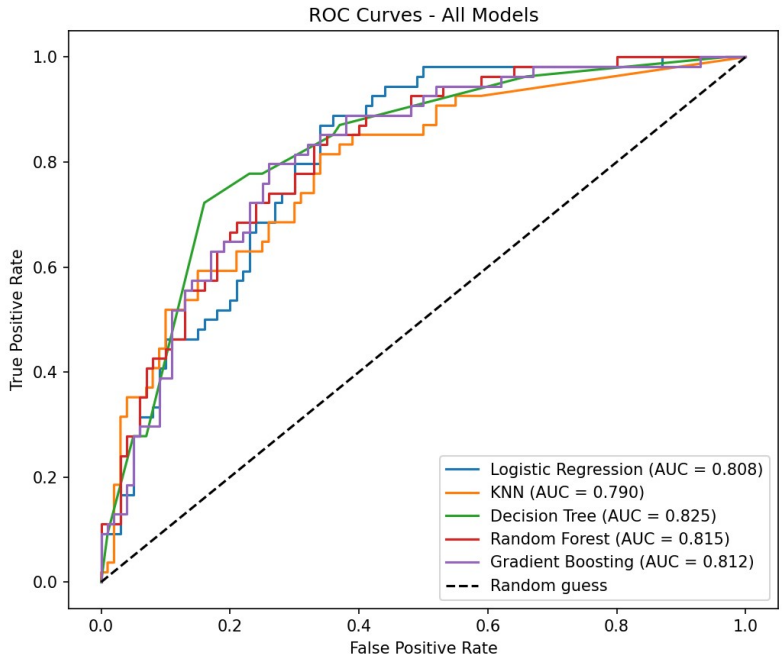


Figure 5.3: ROC curves of all models overlaid on one graph (test set)

5.5 Feature Importance (Random Forest)

Figure 5.4 shows that Glucose is by far the most important feature (0.356), followed by BMI (0.173) and Age (0.126), then DiabetesPedigreeFunction (0.094), Insulin (0.074), Pregnancies (0.064), BloodPressure (0.056) and SkinThickness (0.056). This agrees with the correlation analysis (Figure 2.3) and with clinical knowledge that blood glucose, body weight and age are the main risk indicators. Impurity-based importances can favour continuous features and are less reliable for Insulin and SkinThickness, where many values were imputed.

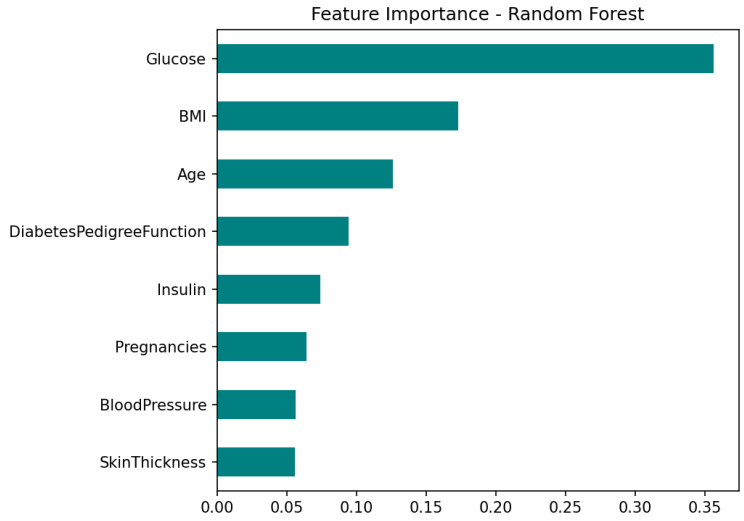


Figure 5.4: Feature importance of the selected Random Forest

5.6 Cross-Validated Performance (Table 5.3)

Because the test set has only 154 samples, a single split is a noisy estimate: the standard error of an accuracy near 75% on 154 samples is about ±3.5 percentage points. Table 5.3 therefore reports the mean ± standard deviation over 5 stratified folds, which uses all 768 records for both training and validation.

Table 5.3: 5-fold stratified cross-validation results (mean ± std)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	**0.7721 ± 0.0189**	**0.7129 ± 0.0584**	0.5934 ± 0.0392	**0.6450 ± 0.0220**	**0.8395 ± 0.0170**
KNN	0.7487 ± 0.0247	0.6579 ± 0.0376	0.5822 ± 0.0483	0.6173 ± 0.0419	0.7874 ± 0.0217
Decision Tree	0.7343 ± 0.0401	0.6256 ± 0.0643	**0.6493 ± 0.1020**	0.6287 ± 0.0444	0.7845 ± 0.0381
Random Forest	0.7655 ± 0.0287	0.7038 ± 0.0619	0.5745 ± 0.0676	0.6298 ± 0.0518	0.8358 ± 0.0197
Gradient Boosting	0.7538 ± 0.0210	0.6593 ± 0.0449	0.6232 ± 0.0652	0.6372 ± 0.0366	0.8223 ± 0.0255

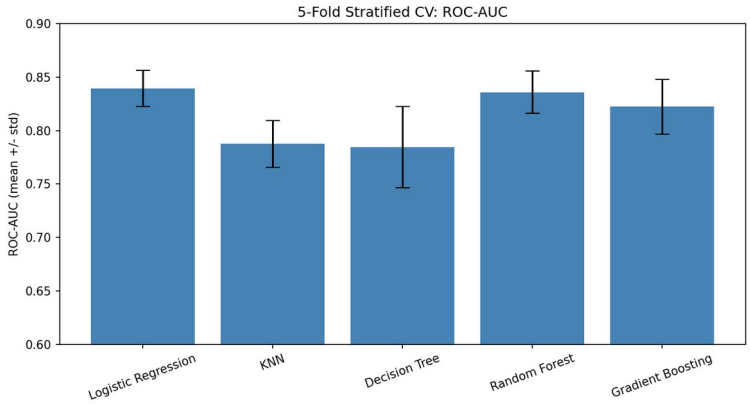


Figure 5.5: Cross-validated ROC-AUC (mean ± standard deviation) per model

CHAPTER 6
ANALYSIS & INTERPRETATION

6.1 Best and Worst Model per Metric

Table 6.1: Best and worst model for each metric (DT = Decision Tree, LR = Logistic Regression, RF = Random Forest)

Metric	Test set: best	Test set: worst	5-fold CV: best	5-fold CV: worst
Accuracy	DT (0.7987)	LR (0.7143)	LR (0.7721)	DT (0.7343)
Precision	DT (0.7091)	LR (0.6087)	LR (0.7129)	DT (0.6256)
Recall	DT (0.7222)	LR & RF (0.5185)	DT (0.6493)	RF (0.5745)
F1-Score	DT (0.7156)	LR (0.5600)	LR (0.6450)	KNN (0.6173)
ROC-AUC	DT (0.8248)	KNN (0.7904)	LR (0.8395)	DT (0.7845)

6.2 Why the Rankings Differ between the Test Set and Cross-Validation

On the single test split the Decision Tree is best on every metric, which seems surprising because it is the simplest tree-based model. The cross-validation results show that this is a split-specific effect rather than genuine superiority: its cross-validated ROC-AUC (0.785) is the lowest of all models and it has the largest standard deviation (±0.038; its recall varies by ±0.102 between folds). A single tree is a high-variance learner. On the test set it misclassified 31 of 154 patients versus 38–44 for the other models, a difference within the sampling error of a 154-sample test set.

Under cross-validation the models separate into two groups: Logistic Regression (0.839), Random Forest (0.836) and Gradient Boosting (0.822) form the top group, while KNN (0.787) and Decision Tree (0.785) are clearly behind. The gap between Logistic Regression and Random Forest (0.004) is far smaller than their standard deviations (0.017 and 0.020), so they are statistically tied.

6.3 Why Ensemble Models Tend to Perform Better

- **Variance reduction (bagging):** Random Forest averages hundreds of decorrelated trees, so the errors of individual trees cancel out. This is visible here: Random Forest improves cross-validated ROC-AUC over a single Decision Tree by 0.051 (0.836 vs 0.785) and halves its fold-to-fold standard deviation (0.020 vs 0.038).
- **Bias reduction (boosting):** Gradient Boosting adds trees that specifically correct the remaining errors, which allows it to model complex interactions; here it achieved higher recall (0.623) than Random Forest (0.575) in cross-validation.

It is important to note that in this project the ensembles **did not beat** Logistic Regression. With only 768 records and a few almost monotonic features (higher Glucose and BMI imply higher risk), a regularised linear model is already close to the best boundary, so ensembles have little extra structure to exploit. They do, however, clearly beat the single tree and KNN.

6.4 Interpretation for Screening

In cross-validation every model has recall between 0.57 and 0.65, so at the default 0.5 threshold roughly 35–43% of diabetic patients are missed. In screening this is the key weakness, more relevant than the small accuracy differences between models. Lowering the decision threshold, class weights or oversampling can trade some precision for higher recall.

CHAPTER 7
BEST MODEL SELECTION

7.1 Justification of the Selected Model

Selected model: Random Forest (n_estimators = 300, max_depth = 6, min_samples_split = 5, min_samples_leaf = 2, max_features = sqrt). The decision was based on several criteria rather than on accuracy alone:

1. **Discrimination ability (ROC-AUC):** cross-validated AUC of 0.836 ± 0.020 is statistically indistinguishable from the highest value (Logistic Regression, 0.839 ± 0.017) and clearly above KNN, Decision Tree and slightly above Gradient Boosting. On the test set its AUC (0.815) is also higher than that of Logistic Regression (0.809), KNN and Gradient Boosting.
2. **Stability:** its fold-to-fold variation is about half that of the Decision Tree, so its performance is more likely to carry over to new patients.
3. **Error profile:** on the test set it makes the fewest false alarms (13 false positives, precision 0.683) and has the highest number of correctly identified healthy patients (87), which limits unnecessary follow-up tests.
4. **Non-linear modelling and interpretability:** unlike Logistic Regression it can capture non-linear effects and interactions between measurements, and it provides feature importances (Figure 5.4) that can be checked against medical knowledge.

The feature-importance ranking (Figure 5.4, Section 5.5) puts Glucose (0.356), BMI (0.173) and Age (0.126) first, in line with the correlation analysis and clinical knowledge.

Why the other models were not selected

- **Logistic Regression:** the strongest competitor, statistically tied on AUC and the simplest model. It would be the better choice if coefficient-level explanations or well-calibrated probabilities were the priority. It was not chosen because it assumes linearity and had the lowest test-set accuracy, precision and F1.
- **Gradient Boosting:** runner-up; higher recall than Random Forest but slightly lower and more variable AUC (0.822 ± 0.026) and more hyperparameters to tune.
- **Decision Tree:** best on the single test split but worst in cross-validation (AUC 0.785 ± 0.038), i.e. unstable.
- **KNN:** lowest test-set AUC (0.790) and no interpretability.

Caveat: Random Forest also has the lowest cross-validated recall (0.575). If the deployment goal were to catch as many diabetic patients as possible, Gradient Boosting or a threshold-tuned Random Forest would be preferable.

The selected Random Forest is deployed as the Diabetes Risk Predictor, described in Sections 7.2 to 7.7.

7.2 Deployment: Diabetes Risk Predictor

To show the trained model in use, the selected Random Forest was packaged as the **Diabetes Risk Predictor**: a single web page (*diabetes_risk_app.html*, about 400 KB) that estimates a patient's probability of diabetes from the same eight measurements used in training. All computation runs in the user's browser, so there is no server or installation and no patient data leaves the device; the page opens by double-clicking the file in any modern browser. A Python (Streamlit) version, *app.py*, and a step-by-step Jupyter notebook,

ML_Mini_Project_Demo.ipynb, were also prepared, and the training script *ml_mini_project.py* regenerates every figure and table of this report.

7.3 Technology Stack

Table 7.1: Technology stack

Component	Technology	Purpose
Languages	Python 3, JavaScript (ES6), HTML5, CSS3	Training, in-browser prediction, user interface
Data and ML	pandas, NumPy, scikit-learn	Cleaning, pipeline, five classifiers, metrics, cross-validation
Visualisation	Matplotlib, Seaborn	EDA and result figures
Model export	Python json module (export_model.py)	Stores 300 trees and preprocessing parameters as model.json (391 KB)
Front end	HTML, CSS, plain JavaScript (no framework)	Input form, live result, feature-importance chart
Demo and testing	Jupyter, Streamlit, Node.js, Playwright (headless Chromium)	Notebook, Python demo app, comparison with scikit-learn, page tests

Hardware requirements: any laptop able to run Python 3 for training and any modern browser (PC or smartphone) for the predictor; no GPU is needed because the model is a small ensemble of shallow trees.

7.4 Flowchart and Architecture

Figure 7.1 shows the workflow: the left lane runs once, offline, in Python, and the right lane runs in the browser every time a user enters patient values. Figure 7.2 shows the two environments; model.json is the single file passed from Python to the web page, and verify.js checks it against scikit-learn.

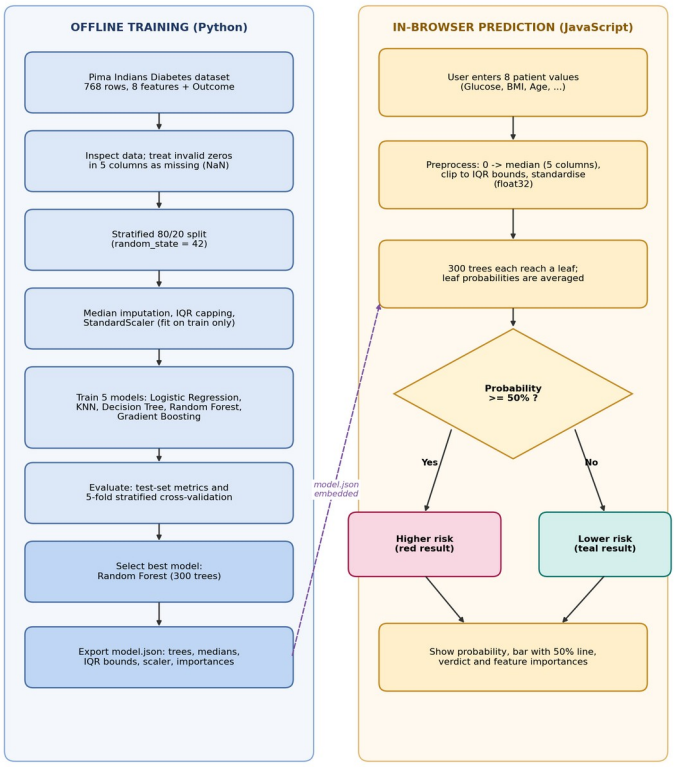


Figure 7.1: Flowchart of the Diabetes Risk Predictor (training and prediction)

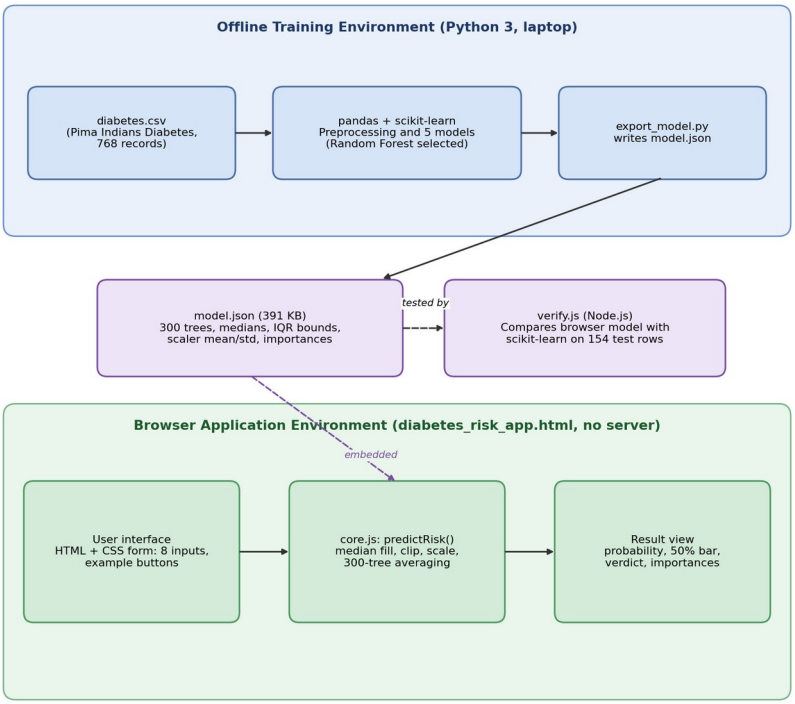


Figure 7.2: Architecture of the Diabetes Risk Predictor

7.5 Core Module: Running the Random Forest in the Browser

A scikit-learn forest cannot run in a browser directly, so it is converted to plain data. For every tree the exporter stores three arrays: *f* (feature index, −1 for a leaf), *t* (split threshold, or the probability of diabetes at a leaf) and *r* (index of the right child). scikit-learn numbers nodes

depth-first, so the left child of node n is always node n + 1 (asserted in the export script) and need not be stored. The 300 trees, with the training medians, IQR bounds, scaler parameters and feature importances, occupy 391 KB as model.json and are embedded in the page. Prediction follows five steps:

- **Missing values:** blank fields, and zeros in Glucose, BloodPressure, SkinThickness, Insulin and BMI, are replaced by the training medians (117, 72, 29, 125 and 32.4); a zero in Pregnancies is valid and kept.
- **Capping and scaling:** each value is clipped to the training IQR bounds, then standardised with the training mean and standard deviation (rounded to 32-bit precision).
- **Tree traversal:** every tree is walked from the root to a leaf by comparing the scaled value with each split threshold.
- **Voting:** the 300 leaf probabilities are averaged, and a probability of 0.5 or more is reported as "higher risk".

The core function, condensed from core.js:

```
function predictRisk(M, raw) {
  const x = raw.map((v, i) => {
    // missing value, or 0 in columns 1-5  ->  training median
    if (v == null || Number.isNaN(v) || (i >= 1 && i <= 5 && v === 0)) v = M.med[i];
    v = Math.min(Math.max(v, M.lo[i]), M.hi[i]);          // IQR capping
    return Math.fround((v - M.mean[i]) / M.scale[i]);     // float32 scaling
  });
  let sum = 0;
  for (const T of M.trees) {                              // 300 trees
    let n = 0;
    while (T.f[n] !== -1)                                 // -1 marks a leaf
      n = x[T.f[n]] <= T.t[n] ? n + 1 : T.r[n];           // left child is n + 1
    sum += T.t[n];                                        // leaf probability
  }
  return sum / M.trees.length;                            // average of all trees
}
```

7.6 User Interface

The page has two panels (Figure 7.3). On the left the user enters the eight measurements, each with its unit; Skin thickness and Insulin accept 0 for "unknown", and two buttons fill in an example low-risk or high-risk patient. The result updates on every keystroke.

- **Result panel:** the probability in large type, a one-line verdict, and a bar with a marked 50% decision line; the colour changes from teal (lower risk) to dark red (higher risk).
- **Feature importance and performance:** horizontal bars from the Random Forest (Glucose 35.6%, BMI 17.3%, Age 12.6%, …) and a box with test accuracy 74.7%, test ROC-AUC 0.815 and 5-fold CV ROC-AUC 0.836.
- **Safety note:** the page states that the model finds only about 57% of diabetic patients in cross-validation and is a class project, not medical advice.

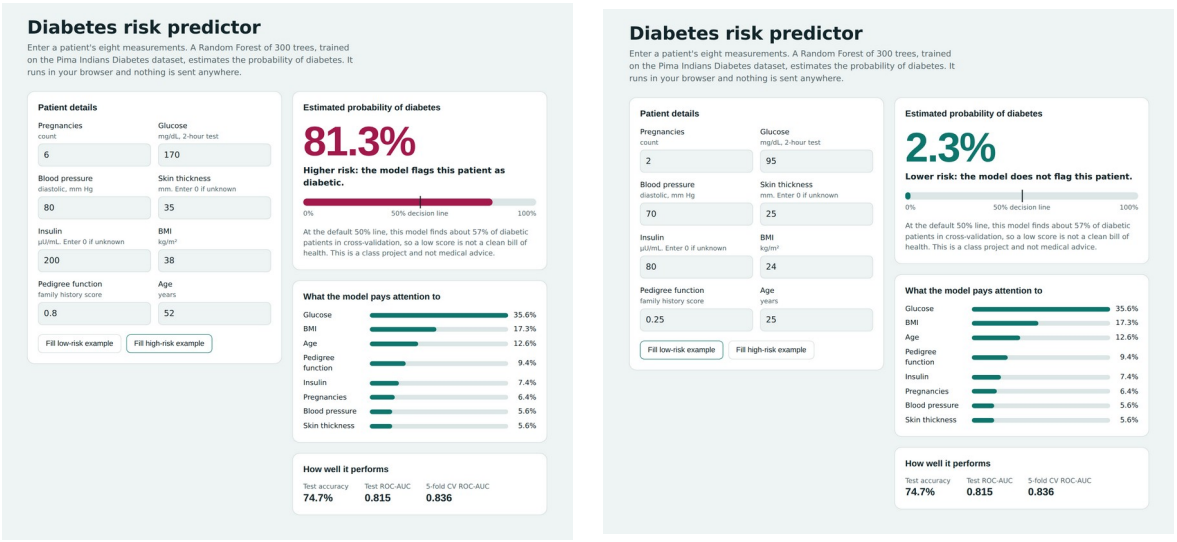


Figure 7.3: Diabetes Risk Predictor with the high-risk example (left, 81.3%) and the low-risk example (right, 2.3%) (screenshots)

7.7 Testing and Verification

The application was tested at three levels: exact agreement with scikit-learn, behaviour of the preprocessing rules, and the user interface in a real browser (headless Chromium). Table 7.2 lists the test cases and observed results.

Table 7.2: Test cases for the Diabetes Risk Predictor

ID	Test	Expected	Observed	Result
T1	Browser model vs scikit-learn on all 154 test patients	Same probabilities	Maximum absolute difference 0; 0 label differences	Pass
T2	High-risk example (6, 170, 80, 35, 200, 38, 0.8, 52)	High probability, flagged	81.3%, higher risk	Pass
T3	Low-risk example (2, 95, 70, 25, 80, 24, 0.25, 25)	Low probability, not flagged	2.3%, lower risk	Pass
T4	Skin thickness = 0 and Insulin = 0 vs the training medians (29 and 125), other values equal	Identical result	54.35% in both cases	Pass
T5	Pregnancies = 0 is kept as a real value (not replaced by the median 3)	Result differs from Pregnancies = 3	19.1% vs 18.1%	Pass
T6	A blank input field	No error; treated as missing	Page keeps working	Pass
T7	Streamlit app (Python) vs browser app on the two examples	Same probabilities	81.3% and 2.3% in both	Pass
T8	Phone width (390 px)	No horizontal scrolling	No overflow	Pass

Figure 7.4 plots the browser probability against the scikit-learn probability for every test patient; all 154 points lie on the line y = x.

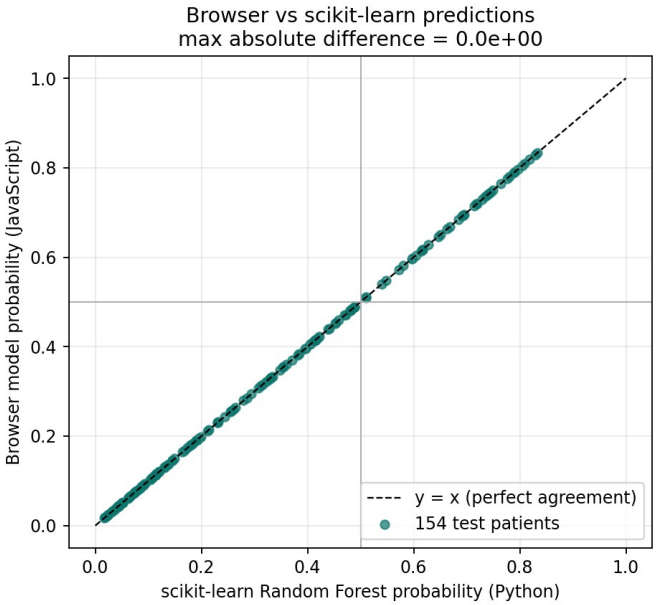


Figure 7.4: Browser model vs scikit-learn Random Forest on the 154 test patients

Exact agreement was not obtained on the first attempt. A first JavaScript version compared values in 64-bit precision and differed from scikit-learn by up to 3.0 percentage points for some patients (no predicted label changed), because scikit-learn converts its inputs to 32-bit floats before comparing them with split thresholds. A value exactly equal to a threshold in 32-bit precision can be marginally larger in 64-bit precision and take the other branch; for the worst patient this affected 12 of the 300 trees. Rounding values with *Math.fround* before the comparison removed the difference completely.

CHAPTER 8
ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE

8.1 Advantages

- Uses only eight inexpensive, routinely available measurements (glucose, BMI, age and others), so it could be applied at a basic health check-up.
- The pipeline is leak-free: imputation, IQR capping and scaling are fitted only on training data and re-fitted inside every cross-validation fold.
- Hidden missing values (zeros in five columns) were detected and handled; this matters because 48.7% of Insulin and 29.6% of SkinThickness values were invalid.
- Random Forest reaches a cross-validated ROC-AUC of about 0.84, and its feature importances (Glucose, BMI, Age) agree with medical knowledge.
- Results are reproducible (random_state = 42) and validated with both a hold-out set and 5-fold cross-validation.
- The Diabetes Risk Predictor is a single 400 KB web page that needs no installation or server, keeps patient values on the device and reproduces the Python model exactly (maximum difference 0 on 154 test patients).

8.2 Limitations

- The dataset has only 768 records, all from women aged 21 or more of one population (Pima Indian heritage), so the models may not generalise to men, other groups or younger patients.
- Almost half of the Insulin values and nearly a third of SkinThickness values were imputed with the median, which reduces the information in those features and may bias the models and their importances.
- Mild class imbalance (34.9% positive) and low recall (57–65% in cross-validation) mean that many diabetic patients would be missed at the default 0.5 threshold.
- Hyperparameters were set manually and not tuned by search, so scores may slightly underestimate what is achievable.
- The deployed model is static, and its probabilities were not checked for calibration: a displayed 81% is a model score, not a guaranteed 81% chance.
- Some inputs (2-hour glucose test, 2-hour serum insulin, skin-fold thickness) need clinical tests, so the application suits clinic-style data, not home use. The study is not clinically validated and must not be used for real diagnosis.

8.3 Future Scope

- Tune hyperparameters with GridSearchCV / RandomizedSearchCV (or Optuna) inside cross-validation.
- Raise recall by tuning the threshold on the precision–recall curve, using class_weight = “balanced”, or SMOTE; add a threshold slider to the Diabetes Risk Predictor.
- Replace median imputation of Insulin and SkinThickness by KNN or MICE imputation, with a “was missing” indicator feature.
- Try XGBoost, LightGBM or stacking; add SHAP explanations to the application and calibration curves to check probability reliability.
- Validate on an independent, larger dataset before any real-world use.

CHAPTER 9
CONCLUSION

This project compared Logistic Regression, KNN, Decision Tree, Random Forest and Gradient Boosting for predicting diabetes on the Pima Indians dataset after careful handling of hidden missing values, outliers and scaling without data leakage. On the single test split the Decision Tree appeared best (accuracy 79.9%, AUC 0.825), but 5-fold cross-validation showed that this was an unstable result: Logistic Regression (AUC 0.839) and Random Forest (AUC 0.836) were the most reliable models, and the ensemble methods clearly outperformed the single tree and KNN.

Random Forest was chosen as the final model because of its high and stable discrimination, low false-positive count and interpretable feature importances, which identified Glucose, BMI and Age as the main predictors. The model was deployed as the browser-based Diabetes Risk Predictor, whose output matches the Python model exactly on all 154 test patients, showing that the trained model can be used without any server. The main weakness of all models is recall (about 57–65%), so the most valuable improvement is to tune the decision threshold or use cost-sensitive learning, together with proper hyperparameter search and validation on larger, more diverse data.

REFERENCES

[1] "Pima Indians Diabetes Database," Kaggle (UCI Machine Learning). [Online]. Available: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

[2] J. Brownlee, "pima-indians-diabetes.data.csv" (identical 768-record copy, header row added by the authors), GitHub. [Online]. Available: https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

[3] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in Proc. Symp. Computer Applications in Medical Care, 1988, pp. 261–265.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/stable/

[5] scikit-learn developers, "Ensemble methods," scikit-learn user guide. [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html

[6] scikit-learn developers, "Model evaluation: quantifying the quality of predictions," scikit-learn user guide. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[7] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[9] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf. (SciPy), 2010, pp. 56–61. [Online]. Available: https://pandas.pydata.org/docs/

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/stable/

[11] M. L. Waskom, "seaborn: Statistical data visualization," J. Open Source Softw., vol. 6, no. 60, p. 3021, 2021. [Online]. Available: https://seaborn.pydata.org/

[12] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/

[13] Mozilla, "Math.fround()," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/fround

[14] OpenJS Foundation, "Node.js documentation." [Online]. Available: https://nodejs.org/docs/latest/api/

[15] Project Jupyter, "Jupyter documentation." [Online]. Available: https://docs.jupyter.org/